

Challenge:	Broken Cipher
Category:	RSA/Wiener's Attack
Difficulty:	Medium
Tools Used:	RsaCtfTool

Summary

The challenge presents an RSA-based encryption setup that appears standard at first glance, with a public key and ciphertext provided. However, an accompanying memo reveals a crucial detail: the private exponent d was intentionally reduced to improve decryption performance. This subtle “optimization” weakens the cryptosystem, as RSA relies on a sufficiently large d to maintain its security. The flaw is not in the visible parameters, but in the underlying choice of a small private exponent.

In brief, the procedure involves recognizing this weakness and identifying it as a case vulnerable to **Wiener's Attack**. By applying continued fraction expansion on $\frac{e}{N}$, possible values of d are derived and tested. Once the correct private exponent is recovered, the ciphertext can be decrypted using standard RSA decryption, and converting the result back into bytes reveals the hidden flag.

Problem Statement

An encrypted transmission intercepted from Dawlish's private infrastructure was accompanied by an internal note - one that reveals more than intended.

In it, he boasts about optimizing his encryption to improve performance, specifically by reducing the computational burden of decryption.

But in cryptography, efficiency often comes at a cost.

The public key and ciphertext appear standard. The flaw lies not in what is visible - but in what was altered behind the scenes.

If his “optimization” compromised the mathematical foundation, the system may be more fragile than it seems.

Recover the weakness. Exploit it. And reveal what was meant to stay hidden.

Steps to Solve

1. The challenge gives RSA values (n , e) and an encrypted file `flag.enc`, and the hint indicates the private exponent **d is small**, which makes the system vulnerable to **Wiener's Attack**.
2. Instead of trying to factor n , directly use Wiener's Attack because it specifically works when d is unusually small.
3. Set up the attack tool by cloning it:

```
rm -rf RsaCtfTool
git clone https://github.com/RsaCtfTool/RsaCtfTool.git
cd RsaCtfTool
```

4. Create a virtual environment to avoid dependency issues:
Activate the environment so all packages install locally:
Install required dependencies:

```
python3 -m venv venv

# 2. Activate it
source venv/bin/activate

# 3. Continue with requirements
pip install -r requirements.txt
pip install -e .
```

- Convert the given raw RSA values into a proper public key file because the tool needs standard key format:

```
(venv) lakshya_khanna@LAPTOP-NB25FOLI:~/RsaCtfTool$ RsaCtfTool --createpub \
-n 1296343377262049274121048020677443293385134044018138520091303845379048462224033124221491136033320723302037262313101471307739058836
7231763965404272450651028797518451205642591298532989921119689457431342094586520790978273081665306320450045833141270771161856671896311
2609223314416446611296570526932038915778572452080419050129583362000776517408348481208193597222470859581866255165727638734016609800785
3613942552714320392494411719062188634561784567635180056693102452607215588064706692663514294318492213006282048681760448952065156410914
6216718923774252141954909532529101354005700900479417108728150291462530052397575180029437 \
-e 4574029680241009192803528667681744084363850192291869509497891657389677393227257328707011810250161695654175593358090983826705882315
216511596856967485906533501522925832699380757236577682225864031328213339740798094994585474484546464279945651639972305871206730553420
4624732785920350576423970714805018722842172464184647507722695493879131544143530785651838793583927448811272684981096351871117651500515
6554720629007098027757590958905471206737440464887017978435673758122615361650807057690504654662272671561969451057419547247402072181246
858677988996861832880311673255315081735604028792720356503074740023807139725599734127761 \
> key.pub
private argument is not set, the private key will not be displayed, even if recovered.
```

- This generates key.pub, which is required to run attacks.
- Run Wiener's attack on the generated public key:

```
private argument is not set, the private key will not be displayed, even if recovered.
(venv) lakshya_khanna@LAPTOP-NB25FOLI:~/RsaCtfTool$ RsaCtfTool --publickey key.pub --attack wiener --private

[*] Testing key key.pub.
[*] Performing wiener attack on key.pub.
17%|██████████| 146/863 [00:00<00:00, 204531.86it/s]
[*] Attack success with wiener method !

Results for key.pub:
```

- The tool applies continued fraction mathematics to recover the private exponent d.
- Since the key is vulnerable, the attack succeeds and outputs the full RSA private key.
- Copy the entire private key from the output including BEGIN and END lines.
- Save the key into a file:


```
nano private.pem
```
- Paste the key and save the file.

```
GNU nano 7.2 private.pem
-----BEGIN RSA PRIVATE KEY-----
MIID/wIBAAKCAQBsK9iLyvA8wbTZbFD4q5lnefNniigyK5WoqdfHYSL2mgA8EoM
bgvswxdkiikZhpZE09otAivfMPoZgMY5uG/+UEF5GSMiJ734cKDLBk8EPBP5Y0u
FHLn2jfiXohtyLEJiwY4mI6cE/6Tu4NYV8NJqCLCTqW0u3svGYSbGWZqwh615mxI
em3nu37cyf9NMtqFipYMSmnu52P/H0nU0DkBzMa6hPh9puLUCuyMTA407qHe2PI
AvhbKy53hLBndmcG0dLf/fBwT004+4CBWx6bHIi06VlF/0B7yXHUaktyXDx29NDR
EJvt4IEbLD4HRu84UqHaYLW7yYoUFRVseo39AoIBACQ7uTAdAnj80wiMCv/u+9+
bKX/kP5M0xzjRLN2exmpYxJkmIF+w5THYL09iI7X/FaqcNAJiVETJzXWLYF6LdVT
zaAxmXr2jA31WB0lj9oDw75pbUcjBR5nk0JMi6lhZZLckfwat+16jnelsxDGpK8z
PIX0LVGB3VZUUA15I5tWlq5kg/f610EPWm/N9G3cKLgQIwJ2+M1vuWQKUZLCoIlG
6+BOJT0Ks/GEVwgM7j15vNC8bfM8Jk4JKhfZ1HmRcjAbo9b/Wc6BWKz8srLi/akp
X9M4yA+TjNPLqGIgShoEKazAbfyHk7ePvQF3WbZhd+kwSHD7V05eo57feTLKXJEC
IQcjtwpA1ecYuuh19WLThd13crFWJym9taL7PwTWcoFCQKqBgQCAC3Zg/JNG3/C4
jEPTVvalOrl6Nc1CtcAGAnt53LmKpgQ4yKc40VBJu2AUWwp9aoZ6Qt0CVVpCjblY
0awfV7hZV3+1qM4Xug7NLiTA+TqGeYi8m+c6oWOKiXL4+ldfQDWjkruSBRa8eNU/
UIyr3cFgU7beAK79JDjIB7cMeMLEcwKBQDNTvwaU09F0r/iMeZGIDSyB88uElth
FzatZKCl7PXn64amvVTx+mQJg1Pmr35qrTr1e8i+S04S/Uo3XuF1xLJrEpF7mx7
0o8t9wCAB0XXINjLpLhXkk5J1G3vXK9tRY9RCnMmcdqLbopGnc/K7TLz7gBpp+eC
ox1zy0jk7NM3zwIhAK03CKDV5xi66HX1YtMd3XdsVYnkb21ovs/BNZygUJBAiEA
o7cKQNXnGLrodFvi0x3dd3KxVicpvbWi+z8E1nkBQkECgYBWoACNX6ym/VVpj+4n
gLE3E2QcL53b30j9qTq0o7P7Uqf3kLxuxMfptEqhVphAGr/0fh5+bLKj6uqz4r/63
B+pZ2+vbWx+H4n+/ktttrj0It70BaJoyODYmUS8YZ3UCqAa+v00z7TbXcuc0/USSp
xGdJsvrBhWhqkX3qocYT8Mi9w==
-----END RSA PRIVATE KEY-----
```

13. Ensure flag.enc is present in the same directory.

14. Use Python to decrypt the ciphertext using the recovered private key:

```
(venv) lakshya_khanna@LAPTOP-NB25F0LI:~/RsaCtfTool$ python3 -c "from Crypto.PublicKey import RSA; from Crypto.Util.number import long_to_bytes; key = RSA.import_key(open('private.pem').read()); c = int(open('flag.enc').read().strip()); print(long_to_bytes(pow(c, key.d, key.n)).decode())"
Cruxhunt{w13n3r5_4tt4ck_d3str0y5_b4d_m4th}
```

15. The script performs RSA decryption using $(c^d \bmod n)$ and converts the result into readable text.

16. The output printed is the flag.

The challenge is solved by exploiting the weakness of a small private exponent, which allows Wiener's Attack to recover the private key and decrypt the ciphertext.